Практика №2. Аудит готового клиент-серверного приложения

Завьялов Андрей 23ИС-41

Ответы на вопросы из шага 1

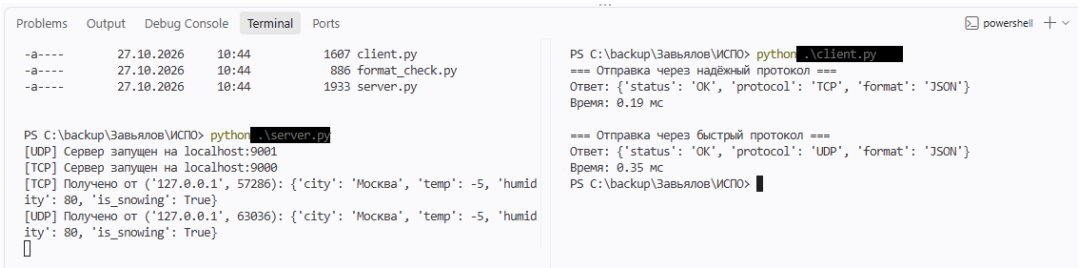
Вопрос 1. Используется TCP. Признаки: SOCK_STREAM — потоковый сокет (только TCP), и вызов connect() устанавливает соединение перед передачей — характерно для TCP.

Вопрос 2. Используется UDP. Признаки: SOCK_DGRAM — датаграммный сокет (UDP), sendto() отправляет пакет без установки соединения.

Вопрос 3. Данные передаются в формате JSON. Подтверждающая строка в client.py: client.send(json.dumps(message, ensure_ascii=False).encode("utf-8")) — json.dumps сериализует словарь в JSON-строку. На сервере это подтверждает json.loads(data.decode("utf-8")).

Вопрос 4. Сервер попытается выполнить json.loads("hello"), получит json.JSONDecodeError, выведет в консоль [TCP] Ошибка: не удалось разобрать JSON от ... и отправит клиенту {"status": "ERROR", "reason": "invalid JSON"}. Сервер не упадёт — ошибка обработана через try/except.

Скриншот с описанием из шага 2



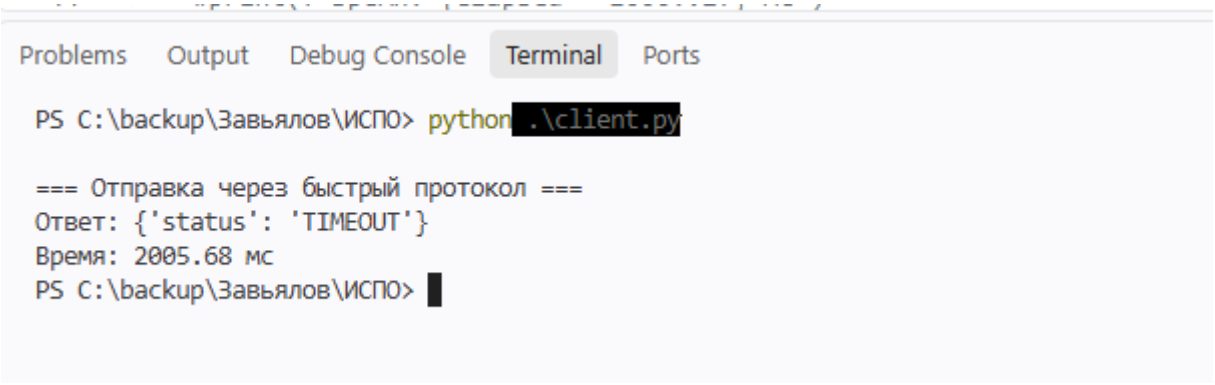
Окно сервера (**слева**): Сервер успешно запустился на портах 9000 (TCP) и 9001 (UDP) и после запуска клиента вывел в консоль полученные от него сообщения с данными о погоде по обоим протоколам.

Окно клиента (**справа**): Клиент получил ответы {'status': 'OK' по TCP (0.19 мс) и UDP (0.35 мс). Оба запроса отработали успешно.

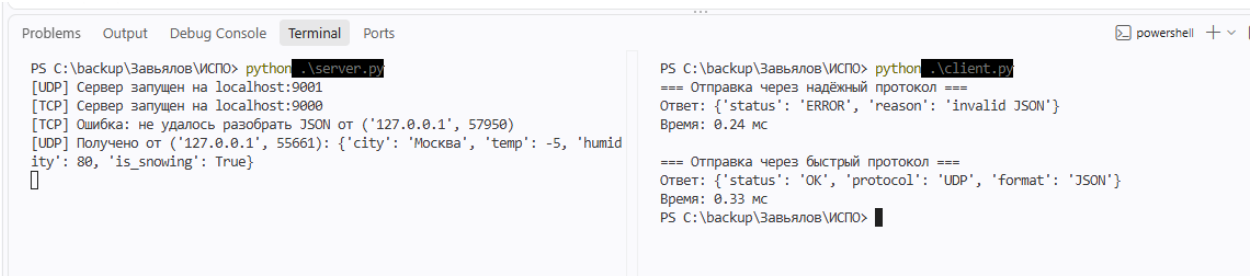
Результаты проверки из шага 3

```
PS C:\backup\Завьялов\ИСПО> python .\client.py
=== Отправка через надёжный протокол ===
Traceback (most recent call last):
  File "C:\backup\Завьялов\ИСПО\client.py", line 42, in <module>
    response, elapsed = send_reliable(weather)
                        ~~~~~~~~~~~~~^^^^^^^^^
  File "C:\backup\Завьялов\ИСПО\client.py", line 18, in send_reliable
    client.connect((HOST, PORT_R))
    ~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^
ConnectionRefusedError: [WinError 10061] Подключение не установлено, т.к. конечный компьютер отверг запрос на подключение
PS C:\backup\Завьялов\ИСПО>
```

Случай 1. Сервер выключен, TCP-клиент. При запуске клиента без сервера возникла ошибка ConnectionRefusedError: [WinError 10061]. Это произошло потому, что TCP требует установки соединения, а метод connect() не смог достучаться до порта 9000, так как его никто не слушает. Ошибка появилась мгновенно, без ожидания.



Случай 2. Сервер выключен, UDP-клиент. Клиент не упал, а «завис» на 2 секунды и вывел {'status': 'TIMEOUT'}. Время выполнения составило ~2005 мс. Причина: UDP не устанавливает соединение, поэтому sendto() просто отправил датаграмму «в никуда». Ответа не последовало, сработал таймаут сокета (settimeout(2.0)), и функция вернула статус TIMEOUT.



Случай 3. Клиент отправляет не JSON (b"hello"). В коде клиента строка сериализации JSON была временно заменена на отправку обычного текста b"hello". Сервер вывел в консоль сообщение [TCP] Ошибка: не удалось разобрать JSON..., но не завершил работу. Клиент получил ответ {'status': 'ERROR', 'reason': 'invalid JSON'}. Причина: строка не является валидным JSON, json.loads() выбросил исключение JSONDecodeError, которое было перехвачено блоком try/except.

```
PS C:\backup\Завьялов\ИСПО> python .\format_check.py
JSON: {"city": "Москва", "temp": -5, "humidity": 80, "is_snowing": true}
Размер JSON: 72 байт
XML: <weather><city>Москва</city><temp>-5</temp><humidity>80</humidity><is_sn
owing>True</is_snowing></weather>
Размер XML: 111 байт

Восстановлено из JSON: {'city': 'Москва', 'temp': -5, 'humidity': 80, 'is_sno
wing': True}
Восстановлено из XML: {'city': 'Москва', 'temp': '-5', 'humidity': '80', 'is_
snowing': 'True'}
Данные совпадают: True
PS C:\backup\Завьялов\ИСПО>
```

Случай 4. Сравнение форматов. JSON оказался компактнее XML (72 байта против 111 байт). Это связано с тем, что в XML каждый ключ оборачивается в открывающий и закрывающий теги, что создаёт избыточность. Кроме того, при восстановлении из XML все значения стали строками ('-5', 'True'), тогда как JSON сохранил исходные типы данных (числа и boolean). Проверка restored_json == weather показала True.

Вывод

Для этого приложения я бы выбрал **протокол TCP** и **формат JSON**.
TCP выбран потому, что он гарантирует доставку данных без потерь и искажений, а для передачи данных о погоде это важно. UDP быстрее, но он не гарантирует, что пакет дойдёт, и не проверяет целостность.
JSON выбран потому, что он компактнее XML (меньше размер передаваемых данных) и при обратном преобразовании сохраняет правильные типы данных (числа остаются числами, а не превращаются в строки, как в XML). Кроме того, в Python с JSON работать гораздо проще.